

# IT Equipment Request Lab — Submission Packet

## 1. Requirements doc & policy definitions

### IT Equipment Request System — Requirements & Policy Design

Phase 1 deliverable. This document alone defines request schema, catalog, role-based frequency limits, escalate rules, decision outcomes, and mock employee fixtures. Later phases implement these rules in code; they must not invent policy beyond what is stated here.

---

#### 1. Request schema

Every equipment request carries the following fields:

| Field | Type | Description |  
| employee\_id | string | Stable employee identifier, pattern EMP-### (e.g. "EMP-101"). |  
| role | string | One of Engineering, Sales, Operations, Management. Authoritative role comes from the employee record, not free-text in the request. Free-text role claims are ignored when they conflict with the record. |  
| item\_requested | string | Catalog key for the item (e.g. "laptop", "monitor", "ergonomic\_chair", or a non-standard name such as "standing\_desk"). |  
| reason | string | Free-text justification supplied by the requester. Used only to detect exceptional circumstances when frequency limits are exceeded; it does not override catalog or role rules. |

Natural-language prompts (agent phase) are parsed into this schema before policy evaluation. Missing employee\_id or item\_requested is invalid input and must not produce an approve decision.

---

#### 2. Role-based policy limits

Frequency is measured from the most recent issue date of that item type on the employee's inventory (or from hire / tenure fields when inventory lacks an issue date — see escalate rule 3).

| Role | Laptop | Monitor | Ergonomic chair |  
| Engineering | 1 every 3 years | up to 2 every 2 years | 1 every 5 years |  
| Sales | 1 every 4 years | 1 every 3 years | 1 every 5 years |  
| Operations | 1 every 4 years | 1 every 3 years | 1 every 5 years |  
| Management | 1 every 2 years | 2 every 2 years | 1 every 5 years |

#### Policy interpretation rules

- Within limit means: for the requested item, either the employee has never been issued that item, or enough full years have elapsed since the last issue date to meet the role's minimum interval, and (for monitors under Engineering/Management) current owned count is below the max quantity.
- Exceeds frequency means: the minimum interval has not elapsed (or quantity would exceed the role max).
- Year intervals use calendar years from issued\_date to "today" at evaluation time (mock "today" may

be fixed in fixtures for deterministic tests).

- ergonomic\_chair is a standard catalog item with a simple default of 1 every 5 years for all four roles, so chair requests remain testable even though the official laptop/monitor tables do not list chairs.

Unknown roles (anything outside the four named roles) are a structured error from policy lookup — not approve, deny, or escalate.

---

### 3. Standard vs non-standard catalog

#### Standard items (auto-eligible for approve/deny against policy)

| Item key | Notes |  
| laptop | Primary compute device; frequency by role (table above). |  
| monitor | Display; quantity + frequency by role. |  
| ergonomic\_chair | Seating; 1 every 5 years for all standard roles. |

#### Non-standard items (always escalate)

Examples (non-exhaustive; any item not in the standard list is non-standard):

- Mechanical keyboard
- Ergonomic split keyboard
- Standing desk
- Docking station variants not on the standard list
- Any free-text item that does not normalize to laptop, monitor, or ergonomic\_chair

Non-standard requests must not be approved or denied by automatic policy; they require human review.

---

### 4. Escalate rules (bounded)

The system escalates only when one of these three criteria holds. Escalation must invoke `flag_for_human_review` (employee id, request summary, reason) and return decision `escalate`.

1. Non-standard item — `item_requested` is not on the standard catalog (laptop / monitor / ergonomic\_chair).
2. Early refresh with exceptional justification — request exceeds frequency, but reason credibly claims exceptional circumstances. Recognized signal phrases (case-insensitive substring match is sufficient for the lab): damaged, crushed, stolen, broken screen, client site (as in damaged on client site). Without such justification, an early request is a clear deny, not escalate.
3. Missing or conflicting tenure / issue-date data — employee record exists but inventory issue dates are missing, malformed, or conflict with tenure fields so eligibility cannot be computed deterministically. Do not guess timelines; escalate.

Borderline timing without exceptional justification is deny, not escalate. Contractors, cost caps, and manager overrides are out of scope for this lab.

---

### 5. Decision outcomes

Every evaluated request ends in exactly one of:

| Outcome | When |

| approve | Item is standard; employee found; role known; within quantity/frequency limits; tenure/issue data sufficient. |

| deny | Item is standard; employee found; role known; exceeds frequency (or would exceed quantity) and reason does not contain exceptional justification. |

| escalate | Any of the three bounded escalate rules; after calling `flag_for_human_review`. |

Supporting structured results (not final outcomes) include:

- Employee not found / invalid id → structured error (tests & tools); agent must not invent an approve.
- Unknown role on record → structured error from `get_policy_limits`.

Eligibility helpers may expose flags such as `eligible`, `decision_hint` (approve / deny / escalate), `exceeds_frequency`, `last_issued`, `next_eligible_date`, and human-readable reasons. They do not inspect free-text justification. The authoritative final outcome comes from `evaluate_request(employee_id, item, reason)`, which applies exceptional-justification phrases on top of eligibility and returns `{decision, rule}`.

---

## 6. Mock employee fixtures (seed for scenarios & tests)

Reference “today” for age calculations in design and tests: 2026-10-02.

| ID | Role | Inventory / history | Purpose |

| EMP-101 | Engineering | Laptop issued 2022-09-01 (~4 years ago) | Clear approve — Engineering laptop ≥ 3 years; standard replacement reason. |

| EMP-102 | Sales | Laptop issued 2026-04-01 (~6 months ago) | Clear deny — Sales laptop interval is 4 years; reason has no damage/exception claim. |

| EMP-103 | Engineering | Laptop issued 2025-10-01 (~1 year ago) | Escalation A — early vs 3-year Engineering rule, but reason claims crushed / stolen / broken screen → must `flag_for_human_review` → escalate. |

| EMP-104 | Management | Monitor ×1 issued 2024-10-01; chair none | Supporting monitor/chair cases (within Management limits when applicable). |

| EMP-105 | Engineering | Laptop present but `issued_date` missing/conflicting with tenure | Tenure-data escalate (optional extra / unit tests). |

| EMP-999 | — | Absent from DB | Not found — supporting edge in unit tests. |

### Target graded scenarios (Phase 6 evidence)

1. Clear approval — EMP-101 requests laptop with a normal wear/performance reason → `evaluate_request` → approve (within\_policy). Must not escalate.
  2. Clear denial — EMP-102 requests laptop replacement after ~6 months with no exceptional justification → `evaluate_request` → deny (exceeds\_frequency). Must not escalate.
  3. Escalation A — EMP-103 early laptop + exceptional reason → `evaluate_request` → escalate (early\_refresh\_exceptional) → call `flag_for_human_review` → escalate.
  4. Escalation B — Any seeded employee (e.g. EMP-101 or EMP-104) requests non-standard gear (e.g. ergonomic split keyboard or standing desk) → `evaluate_request` → escalate (non\_standard\_item) → call `flag_for_human_review` → escalate.
- 

## 7. Escalation queue (side effect)

flag\_for\_human\_review(employee\_id, request, reason) appends a record:

```
[json]
{
  "timestamp": "<ISO-8601 UTC>",
  "employee_id": "EMP-103",
  "request": "laptop",
  "reason": "<human-readable escalation reason>"
}
```

Storage for the lab: in-memory list for the process lifetime, plus optional append to data/escalations.jsonl. Return a confirmation dict including the written fields.

---

## 8. Plain-function + MCP surface (implemented)

These functions encode this document in business\_logic.py and are exposed as MCP tools in server.py.

| Function | Responsibility |  
| get\_employee\_info(employee\_id) | Role, tenure, hardware inventory; structured error if missing/invalid. |  
| get\_policy\_limits(role) | Per-item max quantity + min years; error if unknown role. |  
| check\_request\_eligibility(employee\_id, item) | Compose employee + policy; eligibility hints only (no free-text reason). |  
| evaluate\_request(employee\_id, item, reason) | Authoritative {decision, rule} including exceptional-justification matching. |  
| flag\_for\_human\_review(employee\_id, request, reason) | Queue append + confirmation (required whenever decision is escalate). |

Stable evaluate\_request.rule ids include: within\_policy, exceeds\_frequency, early\_refresh\_exceptional, non\_standard\_item, missing\_tenure.

---

## Done criteria

A grader can reconstruct approve / deny / escalate outcomes for the four scenarios from this document alone, and the agent must copy evaluate\_request.decision rather than inventing policy.

## 2. Minimal one-tool + full-server MCP handshake

Suggested Approach step 3 — trivial ping server/client before business logic:

### Minimal one-tool MCP handshake

```
Connecting to Minimal-Ping-Server over stdio...
=== Handshake (initialize) ===
Protocol: 2025-11-25
Server: Minimal-Ping-Server
Registered tools: ['ping']
ping() -> pong
Minimal one-tool server/client connection confirmed.
```

[text]

```
Connecting to Minimal-Ping-Server over stdio...
=== Handshake (initialize) ===
Protocol: 2025-11-25
Server: Minimal-Ping-Server
Registered tools: ['ping']
ping() -> pong
Minimal one-tool server/client connection confirmed.
```

Full equipment server handshake (python test\_client.py):

```
Full server MCP handshake (test_client.py)
Connecting to IT-Equipment-Server over stdio...

=== Handshake (initialize) ===
Protocol: 2025-11-25
Server: IT-Equipment-Server
Version: 1.30.0

=== Tools discovered (list_tools) ===
- get_employee_info: Look up an employee record from the mock IT equipment database.
- find_employee: Resolve EMP-## from a person's name and optional department/role.
- get_policy_limits: Fetch per-item quantity and frequency limits for a job role.
- check_request_eligibility: Evaluate whether an employee may receive a requested equipment item.
- evaluate_request: Apply the full policy procedure and return {decision, rule}.
- flag_for_human_review: Queue an equipment request for human review (required for all escalations).

Registered tools OK: ['check_request_eligibility', 'evaluate_request', 'find_employee', 'flag_for_human_review', 'get_employee_info', 'get_policy_limits']

=== call_tool("get_employee_info", {"employee_id": "EMP-101"}) ===
{
  "ok": true,
  "employee_id": "EMP-101",
  "name": "Alex Rivera",
  "role": "Engineering",
  "hire_date": "2019-03-15",
  "inventory": [
    {
      "item": "Laptop",
      "issued_date": "2022-09-01",
      "asset_tag": "LT-101"
    }
  ]
}

MCP client smoke test complete.
```

### 3. Full MCP server implementation

[python]

```
'''
This script starts the IT-Equipment-Server over stdio.
1. Initializes the connection
2. Lists the tools
3. Calls the get_employee_info() tool
4. Prints the result
'''

from __future__ import annotations

from typing import Any

from mcp.server.fastmcp import FastMCP

import business_logic as bl

mcp = FastMCP("IT-Equipment-Server")

@mcp.tool()
def get_employee_info(employee_id: str) -> dict[str, Any]:
    """Look up an employee record from the mock IT equipment database.

    Problem this solves: agents need the authoritative role, tenure, and hardware
    inventory before applying replacement policy. Free-text role claims are not trusted.
```

When to call: at the start of an equipment request once you have an `employee_id` (pattern EMP-###), or whenever inventory/role must be re-checked.

Args:

`employee_id`: Employee identifier such as "EMP-101".

Returns:

Success: {ok, `employee_id`, name, role, hire\_date, inventory}.

Failure: {ok: false, error, message, ...} for invalid or missing ids.

"""

`return bl.get_employee_info(employee_id)`

@mcp.tool()

def find\_employee(name: str, role: str = "") -> dict[str, Any]:

"""Resolve EMP-### from a person's name and optional department/role.

Problem this solves: users often identify themselves by name and department instead of `employee_id`. Other tools require EMP-### and must not receive a display name as `employee_id`.

When to call: at the start when the request has a name (e.g. "Alex Rivera") and/or department but no EMP-###. Then call `get_employee_info` with the returned `employee_id`. Claimed department only disambiguates; record role wins.

Args:

`name`: Full or partial employee name, e.g. "Alex Rivera".

`role`: Optional claimed department/role (Engineering, Sales, Operations, Management). Pass "" if unknown.

Returns:

Success: {ok, `employee_id`, name, role, matched\_by, note}.

Failure: {ok: false, error, message, candidates?} if missing/ambiguous.

"""

`return bl.find_employee(name, role or None)`

@mcp.tool()

def get\_policy\_limits(role: str) -> dict[str, Any]:

"""Fetch per-item quantity and frequency limits for a job role.

Problem this solves: eligibility depends on role-specific max quantity and minimum years between issuances for laptop, monitor, and ergonomic\_chair.

When to call: after you know the employee's authoritative role from `get_employee_info`, and before or while interpreting `check_request_eligibility`. Do not invent limits from memory – always read them from this tool.

Args:

`role`: One of Engineering, Sales, Operations, Management (alias "Manager" is normalized to Management).

Returns:

Success: {ok, role, limits: {item: {max\_qty, min\_years}}, standard\_items}.

Failure: {ok: false, error, message} for unknown or empty roles.

"""

`return bl.get_policy_limits(role)`

@mcp.tool()

def check\_request\_eligibility(employee\_id: str, item: str) -> dict[str, Any]:

"""Evaluate whether an employee may receive a requested equipment item.

Problem this solves: composes employee inventory with role policy into a deterministic eligibility result (approve / deny / escalate hints) without

duplicating policy math in the agent. Does NOT inspect free-text reason.

When to call: optional diagnostic after resolving employee\_id and item.  
Prefer evaluate\_request for the authoritative approve/deny/escalate decision  
(it applies exceptional-justification phrases on top of this eligibility).

Args:

employee\_id: Employee identifier such as "EMP-101".  
item: Requested item key (e.g. "laptop", "monitor", "ergonomic\_chair",  
or a non-standard name like "standing\_desk").

Returns:

Success shape includes ok, eligible, decision\_hint, exceeds\_frequency,  
last\_issued, next\_eligible\_date, owned\_count, reasons, and often policy.  
Failure: structured error if the employee id is invalid or not found.

"""

return bl.check\_request\_eligibility(employee\_id, item)

@mcp.tool()

def evaluate\_request(employee\_id: str, item: str, reason: str) -> dict[str, Any]:  
 """Apply the full policy procedure and return {decision, rule}.

Problem this solves: eligibility math alone cannot decide early-refresh cases;  
free-text exceptional justification must be applied deterministically. This  
tool is the authoritative decision – do not invent approve/deny/escalate.

When to call: after you know employee\_id and the requested item, with the  
requester's free-text reason. Then:

- decision=approve or deny → draft Final Decision with that decision
- decision=escalate → call flag\_for\_human\_review, then Final Decision escalate

Args:

employee\_id: Employee identifier such as "EMP-101".  
item: Requested item key (e.g. "laptop", "standing\_desk").  
reason: Free-text justification from the requester.

Returns:

Success: {ok, decision, rule, eligibility, reasons,  
has\_exceptional\_justification, next\_action?}.  
Failure: ok=false with rule/error when lookup fails or reason is empty.

"""

return bl.evaluate\_request(employee\_id, item, reason)

@mcp.tool()

def flag\_for\_human\_review(employee\_id: str, request: str, reason: str) -> dict[str, Any]:  
 """Queue an equipment request for human review (required for all escalations).

Problem this solves: bounded ambiguity must not be resolved by guessing –  
non-standard gear, early refresh with exceptional justification, or missing /  
conflicting tenure data need a human. This tool records the escalation.

When to call: when evaluate\_request returns decision=escalate (or next\_action  
is flag\_for\_human\_review). Call before Final Decision: escalate. Do not call  
for clear approve or clear deny paths.

Args:

employee\_id: Employee identifier such as "EMP-103".  
request: Short request summary (usually the item key, e.g. "laptop").  
reason: Why human review is required (policy conflict, damage claim, etc.).

Returns:

Success: {ok, flagged, queue\_size, entry: {timestamp, employee\_id, request, reason}, message}.  
Failure: {ok: false, error, message} if required fields are empty.

```

"""
return bl.flag_for_human_review(employee_id, request, reason)

if __name__ == "__main__":
    mcp.run(transport="stdio")

```

## 4. Unit test suite and passing terminal logs

### `test\_business\_logic.py`

```

[python]
'''
This script contains the unit tests for the business logic.
1. get_employee_info()
2. get_policy_limits()
3. check_request_eligibility()
4. evaluate_request()
5. flag_for_human_review().
'''

from __future__ import annotations

import json
from datetime import datetime

import pytest

import business_logic as bl

@pytest.fixture(autouse=True)
def _clean_escalations():
    """Clear the escalation queue and truncate the file."""
    bl.clear_escalation_queue(truncate_file=True)
    yield
    bl.clear_escalation_queue(truncate_file=True)

# --- get_employee_info ---

def test_get_employee_info_valid_engineering():
    """EMP-101: valid engineering employee."""
    result = bl.get_employee_info("EMP-101")
    assert result["ok"] is True
    assert result["employee_id"] == "EMP-101"
    assert result["role"] == "Engineering"
    assert result["hire_date"] == "2019-03-15"
    assert any(i["item"] == "laptop" for i in result["inventory"])

def test_get_employee_info_nonexistent():
    """EMP-999: nonexistent employee."""
    result = bl.get_employee_info("EMP-999")
    assert result["ok"] is False
    assert result["error"] == "employee_not_found"

def test_get_employee_info_invalid_id():
    """Invalid employee ID: empty string."""

```

```

result = bl.get_employee_info("")
assert result["ok"] is False
assert result["error"] == "invalid_employee_id"

result = bl.get_employee_info("USER-1")
assert result["ok"] is False
assert result["error"] == "invalid_employee_id"

def test_find_employee_by_name_and_role():
    result = bl.find_employee("Alex Rivera", "Engineering")
    assert result["ok"] is True
    assert result["employee_id"] == "EMP-101"
    assert result["role"] == "Engineering"

def test_find_employee_name_only():
    result = bl.find_employee("Jordan Lee")
    assert result["ok"] is True
    assert result["employee_id"] == "EMP-102"

def test_find_employee_not_found():
    result = bl.find_employee("Nobody Here")
    assert result["ok"] is False
    assert result["error"] == "employee_not_found"

# --- get_policy_limits ---

def test_get_policy_limits_engineering():
    """Engineering role policy limits."""
    result = bl.get_policy_limits("Engineering")
    assert result["ok"] is True
    assert result["limits"]["laptop"] == {"max_qty": 1, "min_years": 3}
    assert result["limits"]["monitor"] == {"max_qty": 2, "min_years": 2}
    assert result["limits"]["ergonomic_chair"] == {"max_qty": 1, "min_years": 5}

def test_get_policy_limits_unknown_role():
    """Unknown role: Intern."""
    result = bl.get_policy_limits("Intern")
    assert result["ok"] is False
    assert result["error"] == "unknown_role"

# --- check_request_eligibility: approve / deny / escalate hints ---

def test_eligible_approve_engineering_laptop_four_years():
    """EMP-101: laptop issued 2022-09-01 vs Engineering 3-year rule → approve."""
    result = bl.check_request_eligibility("EMP-101", "laptop")
    assert result["ok"] is True
    assert result["eligible"] is True
    assert result["decision_hint"] == "approve"
    assert result["exceeds_frequency"] is False
    assert result["last_issued"] == "2022-09-01"
    assert result["years_since_last"] >= 3

def test_deny_sales_laptop_six_months():
    """EMP-102: laptop ~6 months ago, Sales 4-year rule → deny / exceeds_frequency."""
    result = bl.check_request_eligibility("EMP-102", "laptop")
    assert result["ok"] is True

```

```

assert result["eligible"] is False
assert result["decision_hint"] == "deny"
assert result["exceeds_frequency"] is True
assert result["last_issued"] == "2026-04-01"
assert result["next_eligible_date"] == "2030-04-01"

def test_exceeds_frequency_early_engineering_for_agent_escalation():
    """EMP-103 early laptop: exceeds_frequency so agent can escalate on damage claim."""
    result = bl.check_request_eligibility("EMP-103", "laptop")
    assert result["ok"] is True
    assert result["eligible"] is False
    assert result["exceeds_frequency"] is True
    assert result["decision_hint"] == "deny"
    assert result["last_issued"] == "2025-10-01"

def test_non_standard_item_escalate_hint():
    """EMP-101: standing desk → escalate, do not guess."""
    result = bl.check_request_eligibility("EMP-101", "standing_desk")
    assert result["ok"] is True
    assert result["eligible"] is False
    assert result["decision_hint"] == "escalate"
    assert result["exceeds_frequency"] is False
    assert "standard catalog" in result["reasons"][0].lower() or "not on the standard" in result["reasons"][0]

def test_missing_tenure_data_escalate():
    """EMP-105: laptop with null issued_date → escalate, do not guess."""
    result = bl.check_request_eligibility("EMP-105", "laptop")
    assert result["ok"] is True
    assert result["eligible"] is False
    assert result["decision_hint"] == "escalate"
    assert "issued_date" in result["reasons"][0].lower() or "missing" in result["reasons"][0].lower()

def test_eligibility_invalid_employee():
    """EMP-999: nonexistent employee."""
    result = bl.check_request_eligibility("EMP-999", "laptop")
    assert result["ok"] is False
    assert result["error"] == "employee_not_found"

def test_management_chair_never_owned_approve():
    """EMP-104: ergonomic chair never owned → approve."""
    result = bl.check_request_eligibility("EMP-104", "ergonomic_chair")
    assert result["ok"] is True
    assert result["eligible"] is True
    assert result["decision_hint"] == "approve"

def test_management_second_monitor_under_max_qty():
    """EMP-104 has 1 monitor; Management max is 2 → approve additional."""
    result = bl.check_request_eligibility("EMP-104", "monitor")
    assert result["ok"] is True
    assert result["eligible"] is True
    assert result["decision_hint"] == "approve"
    assert result["owned_count"] == 1

# --- evaluate_request: authoritative approve / deny / escalate ---

def test_evaluate_approve_engineering_laptop():
    """Graded approve: EMP-101 laptop within 3-year Engineering rule."""

```

```

result = bl.evaluate_request(
    "EMP-101",
    "laptop",
    "My laptop is 4 years old and slow; need a replacement.",
)
assert result["ok"] is True
assert result["decision"] == "approve"
assert result["rule"] == "within_policy"
assert result["eligibility"]["eligible"] is True

def test_evaluate_deny_sales_laptop_no_exception():
    """Graded deny: EMP-102 early laptop with only performance complaints."""
    result = bl.evaluate_request(
        "EMP-102",
        "laptop",
        "Laptop is freezing and old; please replace it.",
    )
    assert result["ok"] is True
    assert result["decision"] == "deny"
    assert result["rule"] == "exceeds_frequency"
    assert result["has_exceptional_justification"] is False

def test_evaluate_escalate_early_refresh_exceptional():
    """Graded escalate A: EMP-103 early laptop + crushed/stolen justification."""
    result = bl.evaluate_request(
        "EMP-103",
        "laptop",
        "Screen was crushed on a client site / stolen bag – broken screen.",
    )
    assert result["ok"] is True
    assert result["decision"] == "escalate"
    assert result["rule"] == "early_refresh_exceptional"
    assert result["has_exceptional_justification"] is True
    assert result["next_action"] == "flag_for_human_review"
    # Eligibility alone still says deny; evaluate_request upgrades on phrases.
    assert result["eligibility"]["decision_hint"] == "deny"
    assert result["eligibility"]["exceeds_frequency"] is True

def test_evaluate_escalate_non_standard_item():
    """Graded escalate B: non-standard catalog item."""
    result = bl.evaluate_request(
        "EMP-101",
        "ergonomic_split_keyboard",
        "Need a split keyboard for RSI.",
    )
    assert result["ok"] is True
    assert result["decision"] == "escalate"
    assert result["rule"] == "non_standard_item"
    assert result["next_action"] == "flag_for_human_review"

def test_evaluate_escalate_missing_tenure():
    """EMP-105 missing issued_date → escalate / missing_tenure."""
    result = bl.evaluate_request(
        "EMP-105",
        "laptop",
        "Need a laptop refresh; unsure when mine was issued.",
    )
    assert result["ok"] is True
    assert result["decision"] == "escalate"
    assert result["rule"] == "missing_tenure"

```

```

def test_evaluate_early_without_exception_stays_deny():
    """EMP-103 early laptop without exceptional phrases stays deny."""
    result = bl.evaluate_request(
        "EMP-103",
        "laptop",
        "It feels slow and I want a newer model.",
    )
    assert result["ok"] is True
    assert result["decision"] == "deny"
    assert result["rule"] == "exceeds_frequency"

def test_evaluate_unknown_employee():
    result = bl.evaluate_request("EMP-999", "laptop", "Need a laptop.")
    assert result["ok"] is False
    assert result["decision"] == "escalate"
    assert result["rule"] == "employee_not_found"

def test_evaluate_requires_nonempty_reason():
    result = bl.evaluate_request("EMP-101", "laptop", "")
    assert result["ok"] is False
    assert result["error"] == "invalid_reason"

def test_has_exceptional_justification_phrases():
    assert bl.has_exceptional_justification("crushed on client site") is True
    assert bl.has_exceptional_justification("just slow") is False
    assert bl.has_exceptional_justification(None) is False

# --- flag_for_human_review side effects ---

def test_flag_for_human_review_queue_and_timestamp():
    """EMP-103: laptop → flag for human review."""
    result = bl.flag_for_human_review(
        "EMP-103",
        "laptop",
        "Screen crushed on client site; early replacement needed",
    )
    assert result["ok"] is True
    assert result["flagged"] is True
    assert result["queue_size"] == 1

    entry = result["entry"]
    assert entry["employee_id"] == "EMP-103"
    assert entry["request"] == "laptop"
    assert "crushed" in entry["reason"].lower()
    # ISO-8601 parseable timestamp
    datetime.fromisoformat(entry["timestamp"])

    assert len(bl.ESCALATION_QUEUE) == 1
    assert bl.ESCALATION_QUEUE[0]["employee_id"] == "EMP-103"

    # JSONL append side effect
    assert bl.ESCALATIONS_PATH.exists()
    lines = [ln for ln in bl.ESCALATIONS_PATH.read_text(encoding="utf-8").splitlines() if ln.strip()]
    assert len(lines) == 1
    saved = json.loads(lines[0])
    assert saved["employee_id"] == "EMP-103"
    assert saved["reason"] == entry["reason"]

```

```
def test_flag_for_human_review_requires_fields():
    """Empty fields: employee_id, request, reason."""
    assert bl.flag_for_human_review("", "laptop", "reason")["ok"] is False
    assert bl.flag_for_human_review("EMP-101", "", "reason")["ok"] is False
    assert bl.flag_for_human_review("EMP-101", "laptop", "")["ok"] is False
    assert bl.flag_for_human_review("EMP-101", "laptop", None)["ok"] is False
```

## pytest terminal log

[text]

```
===== test session starts =====
platform linux -- Python 3.12.14, pytest-9.1.1, pluggy-1.6.0 -- /workspace/mcp/w7_mcp/.venv/bin/python
cachedir: .pytest_cache
rootdir: /workspace/mcp/w7_mcp
plugins: anyio-4.15.1
collecting ... collected 27 items

test_business_logic.py::test_get_employee_info_valid_engineering PASSED [ 3%]
test_business_logic.py::test_get_employee_info_nonexistent PASSED [ 7%]
test_business_logic.py::test_get_employee_info_invalid_id PASSED [ 11%]
test_business_logic.py::test_find_employee_by_name_and_role PASSED [ 14%]
test_business_logic.py::test_find_employee_name_only PASSED [ 18%]
test_business_logic.py::test_find_employee_not_found PASSED [ 22%]
test_business_logic.py::test_get_policy_limits_engineering PASSED [ 25%]
test_business_logic.py::test_get_policy_limits_unknown_role PASSED [ 29%]
test_business_logic.py::test_eligible_approve_engineering_laptop_four_years PASSED [ 33%]
test_business_logic.py::test_deny_sales_laptop_six_months PASSED [ 37%]
test_business_logic.py::test_exceeds_frequency_early_engineering_for_agent_escalation PASSED [ 40%]
test_business_logic.py::test_non_standard_item_escalate_hint PASSED [ 44%]
test_business_logic.py::test_missing_tenure_data_escalate PASSED [ 48%]
test_business_logic.py::test_eligibility_invalid_employee PASSED [ 51%]
test_business_logic.py::test_management_chair_never_owned_approve PASSED [ 55%]
test_business_logic.py::test_management_second_monitor_under_max_qty PASSED [ 59%]
test_business_logic.py::test_evaluate_approve_engineering_laptop PASSED [ 62%]
test_business_logic.py::test_evaluate_deny_sales_laptop_no_exception PASSED [ 66%]
test_business_logic.py::test_evaluate_escalate_early_refresh_exceptional PASSED [ 70%]
test_business_logic.py::test_evaluate_escalate_non_standard_item PASSED [ 74%]
test_business_logic.py::test_evaluate_escalate_missing_tenure PASSED [ 77%]
test_business_logic.py::test_evaluate_early_without_exception_stays_deny PASSED [ 81%]
test_business_logic.py::test_evaluate_unknown_employee PASSED [ 85%]
test_business_logic.py::test_evaluate_requires_nonempty_reason PASSED [ 88%]
test_business_logic.py::test_has_exceptional_justification_phrases PASSED [ 92%]
test_business_logic.py::test_flag_for_human_review_queue_and_timestamp PASSED [ 96%]
test_business_logic.py::test_flag_for_human_review_requires_fields PASSED [100%]

===== 27 passed in 0.01s =====
```

## 5. Agent code (MCP client connection + ReAct loop)

`agent.py`

[python]

```
"""ReAct agent over the IT Equipment MCP server (Ollama + stdio)."""

from __future__ import annotations

import argparse
import asyncio
import json
import os
```

```

import re
import sys
from datetime import datetime, timezone
from pathlib import Path
from typing import Any

from dotenv import load_dotenv
from mcp import ClientSession, StdioServerParameters
from mcp.client.stdio import stdio_client

import llm
from prompts import PARSE_RETRY_HINT, REFLECTION_PROMPT, build_system_prompt

load_dotenv()

ROOT = Path(__file__).resolve().parent
TRACES_DIR = ROOT / "traces"

ACTION_RE = re.compile(
    r"Action\s*:\s*(\{.*\})",
    re.IGNORECASE | re.DOTALL,
)
FINAL_RE = re.compile(
    r"Final\s*Decision\s*:\s*(approve|deny|escalate)\s*",
    re.IGNORECASE,
)
REASON_RE = re.compile(
    r"Reason\s*:\s*(.+) ",
    re.IGNORECASE | re.DOTALL,
)
THOUGHT_RE = re.compile(
    r"Thought\s*:\s*(.+?)(?=\n\s*(?:Action|Final\s*Decision|Reflection)\s*:\s*|\Z)",
    re.IGNORECASE | re.DOTALL,
)
REFLECTION_RE = re.compile(
    r"Reflection\s*:\s*(.+?)(?=\n\s*Final\s*Decision\s*:\s*|\Z)",
    re.IGNORECASE | re.DOTALL,
)

def _max_steps() -> int:
    return int(os.getenv("AGENT_MAX_STEPS", "8"))

def _extract_json_object(text: str) -> dict[str, Any] | None:
    match = ACTION_RE.search(text)
    raw = match.group(1).strip() if match else None
    if not raw:
        start = text.find("{")
        end = text.rfind("}")
        if start == -1 or end == -1 or end <= start:
            return None
        raw = text[start : end + 1]
    try:
        data = json.loads(raw)
    except json.JSONDecodeError:
        try:
            data = json.loads(raw.split("\n")[0])
        except json.JSONDecodeError:
            return None
    if not isinstance(data, dict):
        return None
    return data

```

```

def parse_assistant(text: str) -> dict[str, Any]:
    """Parse Thought/Action or Thought/Final Decision from model output."""
    thought_m = THOUGHT_RE.search(text)
    thought = thought_m.group(1).strip() if thought_m else ""

    final_m = FINAL_RE.search(text)
    if final_m:
        reason_m = REASON_RE.search(text)
        return {
            "kind": "final",
            "thought": thought,
            "decision": final_m.group(1).lower(),
            "reason": (reason_m.group(1).strip() if reason_m else ""),
            "raw": text,
        }

    action_obj = _extract_json_object(text)
    if action_obj:
        tool = action_obj.get("tool") or action_obj.get("name")
        arguments = action_obj.get("arguments") or action_obj.get("args") or {}
        if not isinstance(arguments, dict):
            arguments = {}
        if tool and str(tool).strip().lower().replace("_", " ") in {
            "final decision",
            "finaldecision",
            "decide",
        }:
            decision = (
                arguments.get("decision")
                or arguments.get("Final Decision")
                or arguments.get("result")
                or "approve"
            )
            decision_s = str(decision).lower().strip()
            if decision_s not in {"approve", "deny", "escalate"}:
                decision_s = "approve"
            return {
                "kind": "final",
                "thought": thought,
                "decision": decision_s,
                "reason": str(
                    arguments.get("reason")
                    or arguments.get("Reason")
                    or thought
                    or "Decision inferred from tool-shaped Final Decision action."
                ),
                "raw": text,
            }
        if tool and isinstance(arguments, dict):
            return {
                "kind": "action",
                "thought": thought,
                "tool": str(tool),
                "arguments": arguments,
                "raw": text,
            }

    return {"kind": "invalid", "thought": thought, "raw": text}

def parse_reflection(text: str) -> dict[str, Any]:
    """Parse Reflection / Final Decision / Reason from the reflection call."""
    reflection_m = REFLECTION_RE.search(text)
    final = parse_assistant(text)
    return {

```

```

        "reflection": (reflection_m.group(1).strip() if reflection_m else text.strip()),
        "decision": final.get("decision") if final.get("kind") == "final" else None,
        "reason": final.get("reason") if final.get("kind") == "final" else "",
        "raw": text,
    }
}

def _observation_text(result: Any) -> str:
    parts: list[str] = []
    if getattr(result, "isError", False):
        parts.append("ERROR")
    for block in getattr(result, "content", []) or []:
        text = getattr(block, "text", None)
        if text is None:
            continue
        try:
            parts.append(json.dumps(json.loads(text), indent=2))
        except json.JSONDecodeError:
            parts.append(str(text))
    return "\n".join(parts) if parts else str(result)

def _slug(prompt: str) -> str:
    cleaned = re.sub(r"[^a-zA-Z0-9]+", "-", prompt.strip())[:48].strip("-").lower()
    return cleaned or "request"

def _save_trace(
    prompt: str,
    lines: list[str],
    decision: str | None,
    *,
    path: Path | None = None,
    extra_header: dict[str, str] | None = None,
) -> Path:
    TRACES_DIR.mkdir(parents=True, exist_ok=True)
    if path is None:
        ts = datetime.now(timezone.utc).strftime("%Y%m%dT%H%M%S")
        path = TRACES_DIR / f"{ts}_{_slug(prompt)}.txt"
    else:
        path.parent.mkdir(parents=True, exist_ok=True)

    header = [
        f"prompt: {prompt}",
        f"model: {llm.ollama_model()}",
        f"host: {llm.ollama_host()}",
        f"decision: {decision or 'none'}",
    ]
    if extra_header:
        for key, value in extra_header.items():
            header.append(f"{key}: {value}")
    header.append("")
    path.write_text("\n".join(header + lines) + "\n", encoding="utf-8")
    return path

def _parse_tool_payload(observation: str) -> dict[str, Any] | None:
    """Best-effort parse of a tool observation that is JSON (possibly pretty-printed)."""
    text = observation.strip()
    if not text:
        return None
    try:
        data = json.loads(text)
    except json.JSONDecodeError:
        start = text.find("{")

```

```

        end = text.rfind("{}")
        if start == -1 or end <= start:
            return None
        try:
            data = json.loads(text[start : end + 1])
        except json.JSONDecodeError:
            return None
        return data if isinstance(data, dict) else None

def run_reflection(
    *,
    user_request: str,
    draft: str,
    observations: str,
) -> dict[str, Any]:
    """Second Ollama call: validate or correct the drafted decision."""
    prompt = REFLECTION_PROMPT.format(
        user_request=user_request,
        draft=draft,
        observations=observations or "(no tool observations)",
    )
    raw = llm.chat(
        [
            {
                "role": "system",
                "content": (
                    "You are reviewing an IT equipment agent's draft. "
                    "Correct errors using tool observations only."
                ),
            },
            {"role": "user", "content": prompt},
        ],
        temperature=0.0,
    )
    parsed = parse_reflection(raw)
    return parsed

async def run_agent(
    user_prompt: str,
    *,
    enable_reflection: bool = True,
    inject_draft: str | None = None,
    trace_path: Path | None = None,
    scenario_id: str | None = None,
) -> dict[str, Any]:
    server_params = StdioServerParameters(
        command=sys.executable,
        args=[str(ROOT / "server.py")],
        cwd=str(ROOT),
    )

    trace_lines: list[str] = []
    observations_log: list[str] = []
    tools_called: list[str] = []
    policy_decision: str | None = None
    policy_rule: str | None = None

    def emit(block: str) -> None:
        print(block, flush=True)
        trace_lines.append(block)

    async with stdio_client(server_params) as (read, write):
        async with ClientSession(read, write) as session:

```

```

await session.initialize()
tools = (await session.list_tools()).tools
known = {t.name for t in tools}
system_prompt = build_system_prompt(tools)
messages: list[dict[str, str]] = [
    {"role": "system", "content": system_prompt},
    {"role": "user", "content": user_prompt},
]

emit(f"=== User request ===\n{user_prompt}\n")
emit(
    f"=== MCP tools ({len(tools)}) ===\n"
    + ", ".join(t.name for t in tools)
    + "\n"
)

decision: str | None = None
reason = ""
parse_retries = 0
step = 0

while step < _max_steps():
    step += 1
    emit(f"=== Step {step} ===")
    assistant = llm.chat(messages)
    messages.append({"role": "assistant", "content": assistant})
    parsed = parse_assistant(assistant)

    if parsed["kind"] == "invalid":
        emit(f"Thought: {parsed.get('thought') or '(parse failed)'}")
        emit(f"Raw model output:\n{assistant}")
        if parse_retries < 1:
            parse_retries += 1
            emit("Observation: invalid format – requesting one parse retry\n")
            messages.append({"role": "user", "content": PARSE_RETRY_HINT})
            continue
        decision = "escalate"
        reason = "Agent output unparseable after retry; escalate rather than guess."
        emit(f"Final Decision: {decision}")
        emit(f"Reason: {reason}\n")
        break

    thought = parsed.get("thought") or ""
    if thought:
        emit(f"Thought: {thought}")

    if parsed["kind"] == "final":
        decision = parsed["decision"]
        reason = parsed.get("reason") or ""
        emit(f"Draft Final Decision: {decision}")
        emit(f"Draft Reason: {reason}\n")
        break

    tool = str(parsed["tool"])
    arguments = parsed["arguments"]
    if tool not in known:
        emit(f"Action: {json.dumps({'tool': tool, 'arguments': arguments})}")
        observation = (
            f"Unknown tool {tool!r}. Valid tools: {sorted(known)}. "
            "Final Decision is NOT a tool – never Action tool=deny/approve/"
            "escalate. If evaluate_request already returned, reply with "
            "Final Decision / Reason (or flag_for_human_review if escalate)."
        )
        emit(f"Observation:\n{observation}\n")
        messages.append({"role": "user", "content": observation})

```

```

        continue

    emit(f"Action: {json.dumps({'tool': tool, 'arguments': arguments})}")
    tools_called.append(tool)

    result = await session.call_tool(tool, arguments)
    observation = _observation_text(result)
    observations_log.append(f"{tool}: {observation}")
    emit(f"Observation:\n{observation}\n")

    route_note = ""
    if tool == "evaluate_request":
        payload = _parse_tool_payload(observation)
        if payload and payload.get("decision") in {
            "approve",
            "deny",
            "escalate",
        }:
            policy_decision = str(payload["decision"])
            policy_rule = str(payload.get("rule") or "")
            if policy_decision == "escalate":
                route_note = (
                    f"[ROUTE] evaluate_request → escalate "
                    f"(rule={policy_rule}). "
                    "Next: call flag_for_human_review, then Final Decision: escalate."
                )
            else:
                route_note = (
                    f"[ROUTE] evaluate_request → {policy_decision} "
                    f"(rule={policy_rule}). "
                    f"Do NOT call flag_for_human_review. "
                    f"Next: Final Decision: {policy_decision}."
                )
            emit(route_note + "\n")

        user_obs = f"Observation from {tool}:\n{observation}"
        if route_note:
            user_obs += f"\n{route_note}"
            messages.append({"role": "user", "content": user_obs})
        else:
            decision = policy_decision or "escalate"
            reason = (
                f"Exceeded max steps ({_max_steps()}); "
                f"using evaluate_request={policy_decision!r} "
                f"(rule={policy_rule}) or escalate."
            )
            emit(f"Draft Final Decision: {decision}")
            emit(f"Draft Reason: {reason}\n")

        draft_decision = decision
        draft_reason = reason
        draft_text = f"Final Decision: {decision}\nReason: {reason}"

        if inject_draft:
            emit("=== Injected bad draft (reflection demo) ===")
            emit(inject_draft + "\n")
            draft_text = inject_draft
            injected = parse_assistant(inject_draft)
            if injected.get("kind") == "final":
                draft_decision = injected["decision"]
                draft_reason = injected.get("reason") or draft_reason

        reflection_note = ""
        if enable_reflection and decision:
            emit("=== Reflection ===")

```

```

reflected = run_reflection(
    user_request=user_prompt,
    draft=draft_text,
    observations="\n\n".join(observations_log),
)
emit(reflected["raw"] + "\n")
reflection_note = reflected.get("reflection") or ""
if reflected.get("decision"):
    reflected_decision = reflected["decision"]
    if reflected_decision != draft_decision:
        emit(
            "=== Reflection outcome ===\n"
            f"draft={draft_decision!r} → reflected={reflected_decision!r}\n"
        )
    else:
        emit(
            "=== Reflection outcome ===\n"
            f"draft validated: {draft_decision!r}\n"
        )
    decision = reflected_decision
    reason = reflected.get("reason") or reason

# Decision lock: evaluate_request wins over LLM draft/reflection.
if policy_decision and decision != policy_decision:
    emit(
        "=== Decision lock ===\n"
        f"Snapping {decision!r} → {policy_decision!r} "
        f"(evaluate_request rule={policy_rule}).\n"
    )
    decision = policy_decision
    if policy_rule and policy_rule not in reason:
        reason = (
            f"{reason} [locked to evaluate_request rule={policy_rule}]"
        ).strip()

if (
    decision == "escalate"
    and "flag_for_human_review" not in tools_called
):
    emit(
        "=== Decision lock ===\n"
        "escalate requires flag_for_human_review; "
        "keeping escalate but note missing flag in this run.\n"
    )

emit(f"Final Decision: {decision}")
emit(f"Reason: {reason}\n")
else:
    if policy_decision and decision != policy_decision:
        emit(
            "=== Decision lock ===\n"
            f"Snapping {decision!r} → {policy_decision!r} "
            f"(evaluate_request rule={policy_rule}).\n"
        )
        decision = policy_decision
    emit(f"Final Decision: {decision}")
    emit(f"Reason: {reason}\n")

saved = _save_trace(
    user_prompt,
    trace_lines,
    decision,
    path=trace_path,
    extra_header={
        **({"scenario": scenario_id} if scenario_id else {}),

```

```

        "draft_decision": str(draft_decision),
        "policy_decision": str(policy_decision or ""),
        "policy_rule": str(policy_rule or ""),
        "reflection": "on" if enable_reflection else "off",
    },
)
emit(f"=== Trace saved ===\n{saved}")

return {
    "decision": decision,
    "reason": reason,
    "draft_decision": draft_decision,
    "draft_reason": draft_reason,
    "policy_decision": policy_decision,
    "policy_rule": policy_rule,
    "reflection": reflection_note,
    "tools_called": tools_called,
    "trace_path": str(saved),
    "steps": step,
}

def main() -> None:
    parser = argparse.ArgumentParser(description="IT equipment ReAct agent (Ollama + MCP)")
    parser.add_argument(
        "prompt",
        nargs="?",
        default=(
            "EMP-101: My laptop is about 4 years old and slowing down during builds; "
            "can I get a replacement laptop?"
        ),
        help="Natural-language equipment request",
    )
    parser.add_argument(
        "--reflect",
        action=argparse.BooleanOptionalAction,
        default=True,
        help="Run Phase 6 reflection after the draft (default: on)",
    )
    parser.add_argument(
        "--inject-draft",
        default=None,
        help="Replace draft with this text before reflection (demo correction)",
    )
    parser.add_argument(
        "--trace-path",
        default=None,
        help="Optional path to write the trace file",
    )
    args = parser.parse_args()
    result = asyncio.run(
        run_agent(
            args.prompt,
            enable_reflection=args.reflect,
            inject_draft=args.inject_draft,
            trace_path=Path(args.trace_path) if args.trace_path else None,
        )
    )
    if not result.get("decision"):
        raise SystemExit(1)

if __name__ == "__main__":
    main()

```

## `llm.py`

```
[python]
"""Thin Ollama /api/chat wrapper for the ReAct agent."""

from __future__ import annotations

import os
from typing import Any

import httpx
from dotenv import load_dotenv

load_dotenv()

DEFAULT_HOST = "http://localhost:11434"
DEFAULT_MODEL = "llama3.1:8b"

def ollama_host() -> str:
    return os.getenv("OLLAMA_HOST", DEFAULT_HOST).rstrip("/")

def ollama_model() -> str:
    return os.getenv("OLLAMA_MODEL", DEFAULT_MODEL)

def chat(
    messages: list[dict[str, str]],
    *,
    model: str | None = None,
    temperature: float = 0.1,
    timeout_s: float | None = None,
) -> str:
    """Send a chat completion to Ollama and return the assistant message content."""
    host = ollama_host()
    chosen_model = model or ollama_model()
    if timeout_s is None:
        timeout_s = float(os.getenv("OLLAMA_TIMEOUT_S", "120"))

    payload: dict[str, Any] = {
        "model": chosen_model,
        "messages": messages,
        "stream": False,
        "options": {"temperature": temperature},
    }

    url = f"{host}/api/chat"
    try:
        with httpx.Client(timeout=timeout_s) as client:
            response = client.post(url, json=payload)
            response.raise_for_status()
    except httpx.ConnectError as exc:
        raise RuntimeError(
            f"Cannot reach Ollama at {host}. "
            "Set OLLAMA_HOST (e.g. http://localhost:11434 or "
            "http://host.docker.internal:11434) and ensure the model is pulled."
        ) from exc
    except httpx.HTTPStatusError as exc:
        raise RuntimeError(
            f'Ollama chat failed ({exc.response.status_code}): {exc.response.text}'
        ) from exc

    data = response.json()
```

```

message = data.get("message") or {}
content = message.get("content")
if not content:
    raise RuntimeError(f"Ollama returned empty content: {data!r}")
return str(content).strip()

```

## `prompts.py`

[python]

```

"""System / reflection prompts for the IT equipment ReAct agent."""

```

```

from __future__ import annotations

```

```

from typing import Any

```

```

SYSTEM_PREAMBLE = """You are an IT equipment request agent. You investigate requests with MCP tools
and draft responses. You do NOT invent policy outcomes – evaluate_request is authoritative.

```

```

Decision rules (follow strictly):

```

- Preferred tool path: (find\_employee if no EMP-###) → get\_employee\_info → evaluate\_request  
→ if decision=escalate call flag\_for\_human\_review → Final Decision.
- Copy evaluate\_request.decision exactly for Final Decision (approve | deny | escalate).
- Cite evaluate\_request.rule in the Reason (e.g. within\_policy, exceeds\_frequency, early\_refresh\_exceptional, non\_standard\_item, missing\_tenure).
- For EVERY escalate you MUST call flag\_for\_human\_review before Final Decision.
- NEVER call flag\_for\_human\_review for clear approve or clear deny.
- check\_request\_eligibility / get\_policy\_limits are optional diagnostics only.
- Authoritative role comes from get\_employee\_info / find\_employee, not user text.
- If the user gives a name/department but NO EMP-###, call find\_employee first.  
NEVER pass a person's name as employee\_id.
- Stop as soon as evaluate\_request (and flag when needed) lets you decide.

```

Output format – each turn, reply with EXACTLY one of these two shapes (no markdown fences).

```

```

Final Decision is NOT a tool. Never put Final Decision inside Action JSON.

```

```

Never use Action with a fake tool name like "deny", "approve", or "escalate".

```

```

When you need a tool:

```

```

Thought: <what you still need>

```

```

Action: {"tool": "<one of: find_employee|get_employee_info|get_policy_limits|check_request_eligibility|evaluate_request|flag_for_human_review>", "arguments": {<json args>}}

```

```

When you can finish (no Action line):

```

```

Thought: <brief justification grounded in evaluate_request>

```

```

Final Decision: approve

```

```

Reason: <cite rule + eligibility facts>

```

```

(Use deny or escalate instead of approve when evaluate_request says so.)

```

```

"""

```

```

def format_tools_for_prompt(tools: list[Any]) -> str:

```

```

    """Render MCP tool schemas into a compact block for the system prompt."""

```

```

    lines = ["Available MCP tools:"]

```

```

    for tool in tools:

```

```

        name = getattr(tool, "name", None) or tool.get("name")

```

```

        desc = getattr(tool, "description", None) or tool.get("description") or ""

```

```

        schema = getattr(tool, "inputSchema", None) or tool.get("inputSchema") or {}

```

```

        first = str(desc).strip().splitlines()[0] if desc else ""

```

```

        lines.append(f"- {name}: {first}")

```

```

    if schema:

```

```

        props = schema.get("properties") or {}

```

```

        required = schema.get("required") or []

```

```

    arg_bits = []
    for key, meta in props.items():
        t = (meta or {}).get("type", "any")
        req = "required" if key in required else "optional"
        arg_bits.append(f"{key}:{t}({req})")
    if arg_bits:
        lines.append(f"  args: {' ', '.join(arg_bits)}")
    return "\n".join(lines)

def build_system_prompt(tools: list[Any]) -> str:
    return SYSTEM_PREAMBLE + "\n" + format_tools_for_prompt(tools)

PARSE_RETRY_HINT = (
    "Your previous reply was not valid. Reply again using EXACTLY either:\n"
    'Thought: ...\nAction: {"tool": "get_employee_info", "arguments": {"employee_id": "EMP-101"}}\n'
    "OR (Final Decision is NOT a tool – do not put it in Action JSON):\n"
    "Thought: ...\nFinal Decision: approve\nReason: ..."
)

REFLECTION_PROMPT = """Review your drafted decision against the retrieved tool outputs.
Did the draft match evaluate_request? Did you avoid inventing a different policy outcome?

Rules while reflecting:
- evaluate_request.decision is authoritative. Final Decision MUST equal that decision.
- If the draft conflicts with evaluate_request, CORRECT to match evaluate_request.decision
  and cite evaluate_request.rule.
- If decision=escalate, observations must include flag_for_human_review; if missing, note that.
- Prefer tool observations over the draft. Do not invent dates or policy numbers.
- Improve Reason prose if needed, but do not change the decision away from evaluate_request.

User request:
{user_request}

Draft decision:
{draft}

Tool observations:
{observations}

Reply with EXACTLY:
Reflection: <validation or what you corrected and why>
Final Decision: <approve|deny|escalate>
Reason: <updated or confirmed reason citing evaluate_request.rule>
"""

```

## 6. Verbatim traces (4 graded scenarios)

### Clear Approval (`traces/scenarios/01\_approve.txt`)

[text]

```

prompt: EMP-101: My laptop is about 4 years old and slowing down during builds; can I get a replacement laptop?
model: llama3.1:8b
host: http://host.docker.internal:11434
decision: approve
scenario: 01_approve
draft_decision: approve
policy_decision: approve
policy_rule: within_policy
reflection: on

```

```

=== User request ===
EMP-101: My laptop is about 4 years old and slowing down during builds; can I get a replacement laptop?

=== MCP tools (6) ===
get_employee_info, find_employee, get_policy_limits, check_request_eligibility, evaluate_request, flag_for_human_review

=== Step 1 ===
Thought: Determine the employee's role and check the request eligibility before evaluating the request.
Action: {"tool": "find_employee", "arguments": {"name": "EMP-101", "role": ""}}
Observation:
{
  "ok": false,
  "error": "employee_not_found",
  "message": "No employee found matching name 'EMP-101'",
  "name": "EMP-101",
  "role": null
}

=== Step 2 ===
Thought: The employee's name is actually "EMP-101", not a person's name. Let's try to find the employee's information using the EMP-### code.
Action: {"tool": "get_employee_info", "arguments": {"employee_id": "EMP-101"}}
Observation:
{
  "ok": true,
  "employee_id": "EMP-101",
  "name": "Alex Rivera",
  "role": "Engineering",
  "hire_date": "2019-03-15",
  "inventory": [
    {
      "item": "laptop",
      "issued_date": "2022-09-01",
      "asset_tag": "LT-101"
    }
  ]
}

=== Step 3 ===
Thought: Now that we have the employee's information, let's evaluate the request to see if Alex Rivera is eligible for a replacement laptop.
Action: {"tool": "evaluate_request", "arguments": {"employee_id": "EMP-101", "item": "laptop", "reason": "laptop is 4 years old and slowing down during builds"}}
Observation:
{
  "ok": true,
  "decision": "approve",
  "rule": "within_policy",
  "eligibility": {
    "ok": true,
    "eligible": true,
    "decision_hint": "approve",
    "exceeds_frequency": false,
    "employee_id": "EMP-101",
    "role": "Engineering",
    "item": "laptop",
    "last_issued": "2022-09-01",
    "next_eligible_date": "2025-09-01",
    "owned_count": 1,
    "policy": {
      "max_qty": 1,
      "min_years": 3
    },
    "years_since_last": 4.08,

```

```

    "reasons": [
      "Last laptop issued 2022-09-01; 4.08 years elapsed meets 3-year Engineering minimum."
    ]
  },
  "reasons": [
    "Last laptop issued 2022-09-01; 4.08 years elapsed meets 3-year Engineering minimum."
  ],
  "has_exceptional_justification": false
}

[ROUTE] evaluate_request → approve (rule=within_policy). Do NOT call flag_for_human_review. Next: Final Decision: approve.

=== Step 4 ===
Thought: The evaluate_request tool has determined that Alex Rivera is eligible for a replacement laptop, and the decision is to approve the request.
Draft Final Decision: approve
Draft Reason: within_policy, last laptop issued 2022-09-01; 4.08 years elapsed meets 3-year Engineering minimum.

=== Reflection ===
Reflection: The draft decision conflicted with evaluate_request.decision. The tool observations showed that the employee was not found, which means the request was not properly evaluated. I corrected the draft to match evaluate_request.decision and cited evaluate_request.rule.

Final Decision: deny
Reason: within_policy, employee_not_found (citing evaluate_request.rule)

=== Reflection outcome ===
draft='approve' → reflected='deny'

=== Decision lock ===
Snapping 'deny' → 'approve' (evaluate_request rule=within_policy).

Final Decision: approve
Reason: within_policy, employee_not_found (citing evaluate_request.rule)

```

## Clear Denial (`traces/scenarios/02\_deny.txt`)

```

[text]
prompt: EMP-102: I am in Sales. My laptop is only about 6 months old and I want a second/replacement laptop because mine feels slow in meetings. No damage.
model: llama3.1:8b
host: http://host.docker.internal:11434
decision: deny
scenario: 02_deny
draft_decision: deny
policy_decision: deny
policy_rule: exceeds_frequency
reflection: on

=== User request ===
EMP-102: I am in Sales. My laptop is only about 6 months old and I want a second/replacement laptop because mine feels slow in meetings. No damage.

=== MCP tools (6) ===
get_employee_info, find_employee, get_policy_limits, check_request_eligibility, evaluate_request, flag_for_human_review

=== Step 1 ===
Thought: Determine if the employee is eligible for a replacement laptop and if the request is within policy.
Action: {"tool": "evaluate_request", "arguments": {"employee_id": "EMP-102", "item": "laptop", "reason": "slow performance"}}
Observation:
{
  "ok": true,
  "decision": "deny",

```

```

"rule": "exceeds_frequency",
"eligibility": {
  "ok": true,
  "eligible": false,
  "decision_hint": "deny",
  "exceeds_frequency": true,
  "employee_id": "EMP-102",
  "role": "Sales",
  "item": "laptop",
  "last_issued": "2026-04-01",
  "next_eligible_date": "2030-04-01",
  "owned_count": 1,
  "policy": {
    "max_qty": 1,
    "min_years": 4
  },
  "years_since_last": 0.5,
  "reasons": [
    "Request exceeds frequency: last laptop issued 2026-04-01; only 0.50 years elapsed, policy requires 4 for Sales. Call evaluate_request with the free-text reason to apply exceptional-justification rules."
  ],
  "reasons": [
    "Request exceeds frequency: last laptop issued 2026-04-01; only 0.50 years elapsed, policy requires 4 for Sales. Call evaluate_request with the free-text reason to apply exceptional-justification rules.",
    "No exceptional justification in reason; clear deny under frequency policy."
  ],
  "has_exceptional_justification": false
}

```

[ROUTE] evaluate\_request → deny (rule=exceeds\_frequency). Do NOT call flag\_for\_human\_review. Next: Final Decision: deny.

=== Step 2 ===

Thought: The request is denied due to exceeding the frequency limit for laptops in the Sales role.

Draft Final Decision: deny

Draft Reason: exceeds\_frequency, last laptop issued 2026-04-01, only 0.50 years elapsed, policy requires 4 for Sales.

=== Reflection ===

Reflection:

- The draft decision was correct, but the reason was not entirely accurate. The policy requires 4 years for Sales, but the tool observations indicate that the last laptop was issued 2026-04-01 and only 0.50 years have elapsed. The correct reason should cite evaluate\_request.rule and include the exact policy requirement.

Final Decision: deny

Reason: exceeds\_frequency, policy requires 4 years for Sales, last laptop issued 2026-04-01, only 0.50 years elapsed.

=== Reflection outcome ===

draft validated: 'deny'

Final Decision: deny

Reason: exceeds\_frequency, policy requires 4 years for Sales, last laptop issued 2026-04-01, only 0.50 years elapsed.

## Escalation A — early + damage (`traces/scenarios/03\_escalate\_damage.txt`)

[text]

prompt: EMP-103: I am in Engineering. My laptop was issued about a year ago but the screen was crushed on a client site / stolen bag incident – broken screen. I need an early replacement.

model: llama3.1:8b

host: http://host.docker.internal:11434

decision: escalate

scenario: 03\_escalate\_damage

draft\_decision: escalate

policy\_decision: escalate

policy\_rule: early\_refresh\_exceptional  
reflection: on

=== User request ===

EMP-103: I am in Engineering. My laptop was issued about a year ago but the screen was crushed on a client site / stolen bag incident – broken screen. I need an early replacement.

=== MCP tools (6) ===

get\_employee\_info, find\_employee, get\_policy\_limits, check\_request\_eligibility, evaluate\_request, flag\_for\_human\_review

=== Step 1 ===

Thought: Determine employee's eligibility for early replacement and check policy limits for Engineering role.

Action: {"tool": "evaluate\_request", "arguments": {"employee\_id": "EMP-103", "item": "laptop", "reason": "broken screen"}}

Observation:

```
{
  "ok": true,
  "decision": "escalate",
  "rule": "early_refresh_exceptional",
  "eligibility": {
    "ok": true,
    "eligible": false,
    "decision_hint": "deny",
    "exceeds_frequency": true,
    "employee_id": "EMP-103",
    "role": "Engineering",
    "item": "laptop",
    "last_issued": "2025-10-01",
    "next_eligible_date": "2028-10-01",
    "owned_count": 1,
    "policy": {
      "max_qty": 1,
      "min_years": 3
    },
    "years_since_last": 1.0,
    "reasons": [
      "Request exceeds frequency: last laptop issued 2025-10-01; only 1.00 years elapsed, policy requires 3 for Engineering. Call evaluate_request with the free-text reason to apply exceptional-justification rules."
    ]
  },
  "reasons": [
    "Request exceeds frequency: last laptop issued 2025-10-01; only 1.00 years elapsed, policy requires 3 for Engineering. Call evaluate_request with the free-text reason to apply exceptional-justification rules.",
    "Free-text reason contains exceptional justification; escalate via flag_for_human_review rather than auto-deny."
  ],
  "has_exceptional_justification": true,
  "next_action": "flag_for_human_review"
}
```

[ROUTE] evaluate\_request → escalate (rule=early\_refresh\_exceptional). Next: call flag\_for\_human\_review, then Final Decision: escalate.

=== Step 2 ===

Thought: The request has been escalated due to exceptional justification, so we need to flag it for human review.

Action: {"tool": "flag\_for\_human\_review", "arguments": {"employee\_id": "EMP-103", "request": "laptop replacement due to broken screen", "reason": "early\_refresh\_exceptional"}}

Observation:

```
{
  "ok": true,
  "flagged": true,
  "queue_size": 1,
  "entry": {
    "timestamp": "2026-10-02T18:35:18.505224+00:00",
    "employee_id": "EMP-103",
    "request": "laptop replacement due to broken screen",
    "reason": "early_refresh_exceptional"
  }
}
```

```

},
"message": "Request flagged for human review."
}

=== Step 3 ===
Thought: The request has been flagged for human review, and we can now make a final decision based on the escalate decision from evaluate_request.
Draft Final Decision: escalate
Draft Reason: early_refresh_exceptional

=== Reflection ===
Reflection:
- The draft decision was correct, but I needed to ensure that the decision matched evaluate_request.decision and cite evaluate_request.rule.
- The draft decision was to escalate, which is correct according to evaluate_request.decision. I also needed to include flag_for_human_review in the observations, which was missing.

Final Decision: escalate
Reason: escalate via flag_for_human_review (citing evaluate_request.rule: "next_action": "flag_for_human_review")

=== Reflection outcome ===
draft validated: 'escalate'

Final Decision: escalate
Reason: escalate via flag_for_human_review (citing evaluate_request.rule: "next_action": "flag_for_human_review")

```

## Escalation B — non-standard item (`traces/scenarios/04\_escalate\_nonstandard.txt`)

```

[text]
prompt: EMP-104: I am in Management. Please order an ergonomic split keyboard (or standing desk) – not a standard catalog laptop/monitor/chair.
model: llama3.1:8b
host: http://host.docker.internal:11434
decision: escalate
scenario: 04_escalate_nonstandard
draft_decision: escalate
policy_decision: escalate
policy_rule: non_standard_item
reflection: on

=== User request ===
EMP-104: I am in Management. Please order an ergonomic split keyboard (or standing desk) – not a standard catalog laptop/monitor/chair.

=== MCP tools (6) ===
get_employee_info, find_employee, get_policy_limits, check_request_eligibility, evaluate_request, flag_for_human_review

=== Step 1 ===
Thought: Determine employee's role and check policy limits for non-standard items.
Action: {"tool": "find_employee", "arguments": {"name": "EMP-104", "role": "Management"}}
Observation:
{
  "ok": false,
  "error": "employee_not_found",
  "message": "No employee found matching name 'EMP-104'",
  "name": "EMP-104",
  "role": "Management"
}

=== Step 2 ===
Thought: Since the EMP-### was provided, try to look up the employee's information directly.
Action: {"tool": "get_employee_info", "arguments": {"employee_id": "EMP-104"}}
Observation:

```

```

{
  "ok": true,
  "employee_id": "EMP-104",
  "name": "Taylor Brooks",
  "role": "Management",
  "hire_date": "2018-11-20",
  "inventory": [
    {
      "item": "monitor",
      "issued_date": "2024-10-01",
      "asset_tag": "MN-104"
    }
  ]
}

=== Step 3 ===
Thought: Now that we have the employee's information, let's evaluate the request.
Action: {"tool": "evaluate_request", "arguments": {"employee_id": "EMP-104", "item": "ergonomic split keyboard", "reason": "non_standard_item"}}
Observation:
{
  "ok": true,
  "decision": "escalate",
  "rule": "non_standard_item",
  "eligibility": {
    "ok": true,
    "eligible": false,
    "decision_hint": "escalate",
    "exceeds_frequency": false,
    "employee_id": "EMP-104",
    "role": "Management",
    "item": "ergonomic_split_keyboard",
    "last_issued": null,
    "next_eligible_date": null,
    "owned_count": 0,
    "reasons": [
      "Item 'ergonomic_split_keyboard' is not on the standard catalog (laptop, monitor, ergonomic_chair); requires human review."
    ]
  },
  "reasons": [
    "Item 'ergonomic_split_keyboard' is not on the standard catalog (laptop, monitor, ergonomic_chair); requires human review."
  ],
  "has_exceptional_justification": false,
  "next_action": "flag_for_human_review"
}

[ROUTE] evaluate_request → escalate (rule=non_standard_item). Next: call flag_for_human_review, then Final Decision: escalate.

=== Step 4 ===
Thought: Since the request was escalated, we need to flag it for human review.
Action: {"tool": "flag_for_human_review", "arguments": {"employee_id": "EMP-104", "request": "ergonomic split keyboard", "reason": "non_standard_item"}}
Observation:
{
  "ok": true,
  "flagged": true,
  "queue_size": 1,
  "entry": {
    "timestamp": "2026-10-02T18:35:35.014359+00:00",
    "employee_id": "EMP-104",
    "request": "ergonomic split keyboard",
    "reason": "non_standard_item"
  }
}

```

```

},
"message": "Request flagged for human review."
}

=== Step 5 ===
Thought: The request has been flagged for human review, and we can now make a final decision based on the escalate decision from evaluate_request.
Draft Final Decision: escalate
Draft Reason: non_standard_item

=== Reflection ===
Reflection:
- The draft decision conflicted with evaluate_request.decision, which is authoritative.
- The draft decision was corrected to match evaluate_request.decision, which is escalate.
- The reason was updated to cite evaluate_request.rule, which is non_standard_item.
- Since the decision is escalate, flag_for_human_review was included in the observations.

Final Decision: escalate
Reason: non_standard_item (evaluate_request.rule)

=== Reflection outcome ===
draft validated: 'escalate'

Final Decision: escalate
Reason: non_standard_item (evaluate_request.rule)

```

## 7. Reflection step log

Reflection correction demo (injected bad draft → corrected decision):

```

[text]
prompt: EMP-101: My laptop is about 4 years old and slowing down; can I get a replacement laptop?
model: llama3.1:8b
host: http://host.docker.internal:11434
decision: approve
scenario: 00_reflection_correction
draft_decision: deny
policy_decision: approve
policy_rule: within_policy
reflection: on

=== User request ===
EMP-101: My laptop is about 4 years old and slowing down; can I get a replacement laptop?

=== MCP tools (6) ===
get_employee_info, find_employee, get_policy_limits, check_request_eligibility, evaluate_request, flag_for_human_review

=== Step 1 ===
Thought: Determine employee's role and check if they are eligible for a laptop replacement.
Action: {"tool": "get_employee_info", "arguments": {"employee_id": "EMP-101"}}
Observation:
{
  "ok": true,
  "employee_id": "EMP-101",
  "name": "Alex Rivera",
  "role": "Engineering",
  "hire_date": "2019-03-15",
  "inventory": [
    {
      "item": "laptop",
      "issued_date": "2022-09-01",
      "asset_tag": "LT-101"
    }
  ]
}

```

```

}
}

=== Step 2 ===
Thought: Check if the employee is eligible for a laptop replacement based on the policy limits for their role.
Action: {"tool": "get_policy_limits", "arguments": {"role": "Engineering"}}
Observation:
{
  "ok": true,
  "role": "Engineering",
  "limits": {
    "laptop": {
      "max_qty": 1,
      "min_years": 3
    },
    "monitor": {
      "max_qty": 2,
      "min_years": 2
    },
    "ergonomic_chair": {
      "max_qty": 1,
      "min_years": 5
    }
  },
  "standard_items": [
    "laptop",
    "monitor",
    "ergonomic_chair"
  ]
}

=== Step 3 ===
Thought: Evaluate the request to replace the laptop using the evaluate_request tool.
Action: {"tool": "evaluate_request", "arguments": {"employee_id": "EMP-101", "item": "laptop", "reason": "equipment is about 4 years old and slowing down"}}
Observation:
{
  "ok": true,
  "decision": "approve",
  "rule": "within_policy",
  "eligibility": {
    "ok": true,
    "eligible": true,
    "decision_hint": "approve",
    "exceeds_frequency": false,
    "employee_id": "EMP-101",
    "role": "Engineering",
    "item": "laptop",
    "last_issued": "2022-09-01",
    "next_eligible_date": "2025-09-01",
    "owned_count": 1,
    "policy": {
      "max_qty": 1,
      "min_years": 3
    },
    "years_since_last": 4.08,
    "reasons": [
      "Last laptop issued 2022-09-01; 4.08 years elapsed meets 3-year Engineering minimum."
    ]
  },
  "reasons": [
    "Last laptop issued 2022-09-01; 4.08 years elapsed meets 3-year Engineering minimum."
  ],
  "has_exceptional_justification": false
}

```

[ROUTE] evaluate\_request → approve (rule=within\_policy). Do NOT call flag\_for\_human\_review. Next: Final Decision: approve.

=== Step 4 ===

Thought: The evaluate\_request tool has already determined that the request should be approved, and there is no need to escalate or flag for human review.

Draft Final Decision: approve

Draft Reason: within\_policy, Last laptop issued 2022-09-01; 4.08 years elapsed meets 3-year Engineering minimum.

=== Injected bad draft (reflection demo) ===

Final Decision: deny

Reason: I am guessing the laptop is too new without trusting the tools.

=== Reflection ===

Reflection: The draft decision conflicted with evaluate\_request.decision, which is authoritative. The draft incorrectly stated that the laptop is too new, but the tool observations show that the laptop was issued in 2022, which is within the 3-year limit for the Engineering role.

Final Decision: approve

Reason: The employee is eligible for a replacement laptop as per the "within\_policy" rule, as the 4.08 years elapsed since the last laptop issue meets the 3-year Engineering minimum. (evaluate\_request.rule)

=== Reflection outcome ===

draft='deny' → reflected='approve'

Final Decision: approve

Reason: The employee is eligible for a replacement laptop as per the "within\_policy" rule, as the 4.08 years elapsed since the last laptop issue meets the 3-year Engineering minimum. (evaluate\_request.rule)

Additional reflection validation also appears inside each scenario trace under the === Reflection === sections.

## 8. Green CI pipeline (screenshot)

## ✓ CI unit tests passed (pytest)

CI evidence (local equivalent – push branch for GitHub Actions green screenshot)

Repo: vidhikansara26/w7\_mcp

Workflow: .github/workflows/ci.yml

Job: Unit tests (business logic)

--- workflow ---

name: CI

on:

push:

branches: [main]

pull\_request:

jobs:

test:

name: Unit tests (business logic)

runs-on: ubuntu-latest

steps:

- name: Checkout  
uses: actions/checkout@v4
- name: Set up Python 3.11  
uses: actions/setup-python@v5  
with:  
python-version: "3.11"  
cache: pip
- name: Install dependencies  
run: |  
python -m pip install --upgrade pip  
pip install -r requirements.txt
- name: Run business logic unit tests  
run: pytest test\_business\_logic.py -v

# Intentionally no Ollama / agent / MCP stdio smoke in CI.

--- pytest test\_business\_logic.py -v ---

```
test_business_logic.py::test_deny_sales_laptop_six_months PASSED [ 37%]
test_business_logic.py::test_exceeds_frequency_early_engineering_for_agent_escalation PASSED [ 40%]
test_business_logic.py::test_non_standard_item_escalate_hint PASSED [ 44%]
test_business_logic.py::test_missing_tenure_data_escalate PASSED [ 48%]
test_business_logic.py::test_eligibility_invalid_employee PASSED [ 51%]
test_business_logic.py::test_management_chair_never_owned_approve PASSED [ 55%]
test_business_logic.py::test_management_second_monitor_under_max_qty PASSED [ 59%]
test_business_logic.py::test_evaluate_approve_engineering_laptop PASSED [ 62%]
test_business_logic.py::test_evaluate_deny_sales_laptop_no_exception PASSED [ 66%]
test_business_logic.py::test_evaluate_escalate_early_refresh_exceptional PASSED [ 70%]
test_business_logic.py::test_evaluate_escalate_non_standard_item PASSED [ 74%]
test_business_logic.py::test_evaluate_escalate_missing_tenure PASSED [ 77%]
test_business_logic.py::test_evaluate_early_without_exception_stays_deny PASSED [ 81%]
test_business_logic.py::test_evaluate_unknown_employee PASSED [ 85%]
test_business_logic.py::test_evaluate_requires_nonempty_reason PASSED [ 88%]
test_business_logic.py::test_has_exceptional_justification_phrases PASSED [ 92%]
test_business_logic.py::test_flag_for_human_review_queue_and_timestamp PASSED [ 96%]
test_business_logic.py::test_flag_for_human_review_requires_fields PASSED [100%]
```

===== 27 passed in 0.01s =====